

MySQL的安装与卸载

由于MySQL的官方安装比较繁琐，我们这里使用集成工具安装MySQL：

- 1. 下载小皮面板集成工具
- 2. 安装小皮面板

安装路径不要带有空格与中文

- 3. 打开小皮，启动Mysql5.7

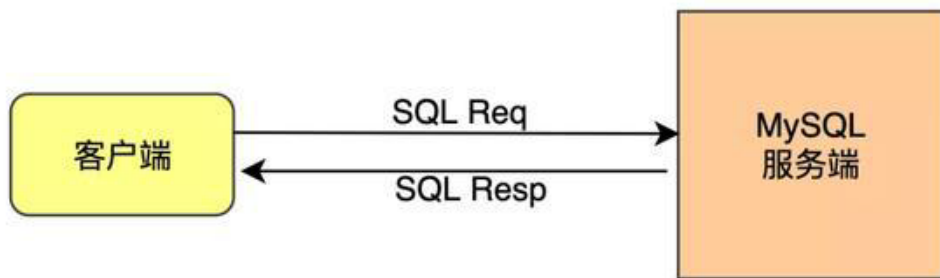


如果出现启动失败的情况，大概率是你的电脑安装过MySQL数据库，此时需要卸载之前安装的MySQL数据库：

- 1. 停止MySQL服务
- 2. 卸载 MySQL 程序：进入控制面板，找到并卸载MySQL，如果有多个 MySQL 相关组件，逐一卸载（例如 MySQL Server、MySQL Workbench等）
- 3. 删除残留文件
 - 打开 MySQL 的安装目录，默认路径通常为 C:\Program Files\MySQL 或 C:\Program Files (x86)\MySQL，删除整个 MySQL 文件夹
 - 打开 MySQL 的数据目录，默认路径为 C:\ProgramData\MySQL（ProgramData 是隐藏文件夹，需启用“显示隐藏文件”），删除 MySQL 文件夹

卸载MySQL并删除残留文件后，小皮即可正常启动MySQL

连接MySQL



MySQL客户端与服务端通过SQL进行交互

之前我们安装的叫做**MySQL服务端**，要想使用MySQL服务，我们需要使用一个**客户端**连接**MySQL服务端**进行使用，我们先来介绍使用MySQL自带的客户端连接MySQL：

1. 找到MySQL的安装路径： 小皮安装路径\MySQL5.7.26\bin
2. 在该路径输入cmd进入命令窗口
3. 登录MySQL

```
mysql -u 用户名 -p  
密码
```

4. 查看MySQL中的所有数据库

```
show databases;
```

实时效果反馈

1.链接MySQL的命令是

- ☒ A sql
- ☐ B ms
- ☐ C plugin
- ☐ D mysql

2.查看MySQL中所有数据库命令是

- ☒ A show databases;
- ☐ B show database;
- ☐ C show dbase;
- ☐ D show data;

答案

1=>D 2=>A

MySQL8_安装



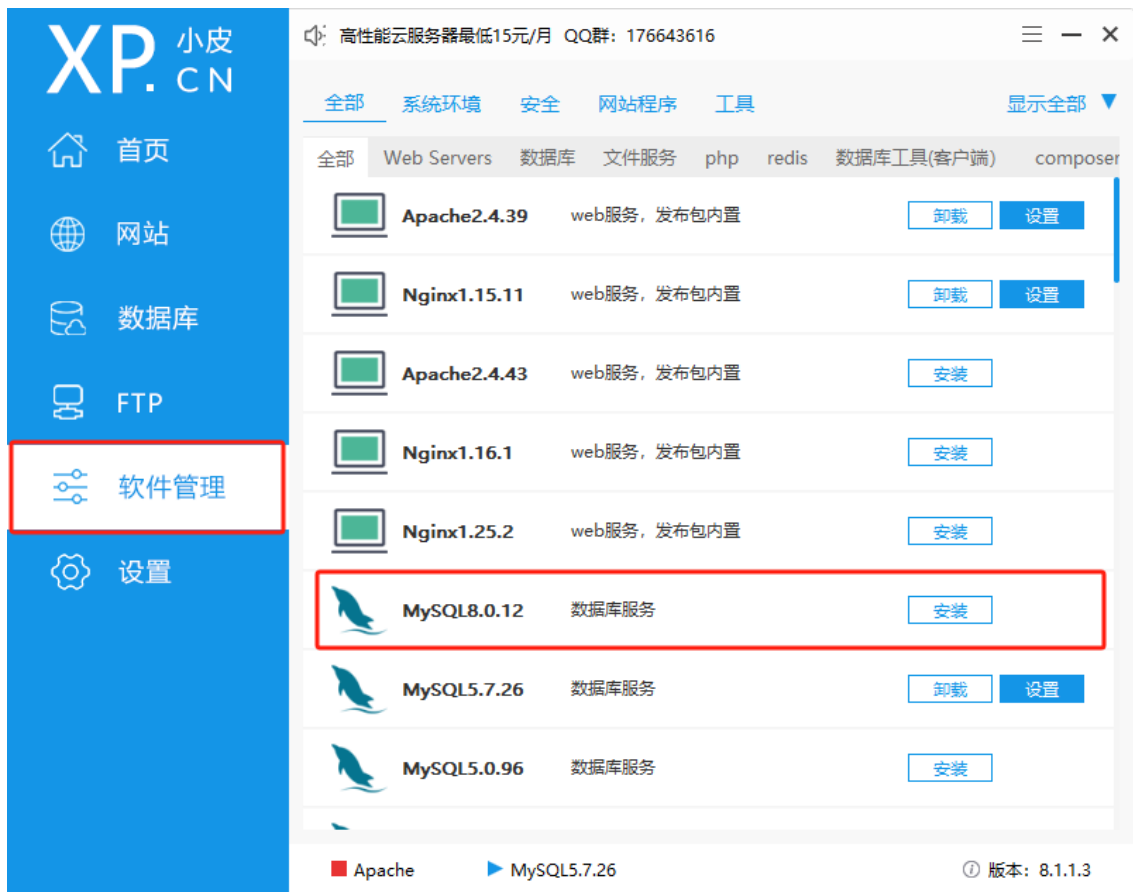
MySQL 8是 MySQL 数据库的一个重要版本，于 2018年4月19日 正式发布，由 Oracle 公司主导开发。它是 MySQL 5.7（2015年发布）之后的一个重大升级版本，跳过了 6.x 和 7.x 版本号，标志着架构、功能和性能的全面革新。

接下来我们在小皮面板中安装MySQL 8.0:

1. 停止MySQL5.7的服务

The screenshot shows the Xp.cn control panel interface. On the left is a blue sidebar with navigation links: 首页 (Home), 网站 (Website), 数据库 (Database), FTP, 软件管理 (Software Management), and 设置 (Settings). The main content area has a top bar with '高性能云服务器最低15元/月' and 'QQ群: 176643616'. Below this is a '一键启动' (One-click Start) section with buttons for 'WNMP' (stopped), '开机自启' (disabled), and '数据库工具' (open). The '套件' (Packages) section lists several services: Apache2.4.39, FTP0.9.60, MySQL5.7.26, and Nginx1.15.11. The MySQL5.7.26 row is highlighted with a red box, and its '停止' (Stop) button is active. Below the packages list is a log section with tabs for '运行状态' (Running Status), '资源信息' (Resource Information), and '日志文件' (Log Files). The '运行状态' tab shows a list of MySQL operations with timestamps and status (e.g., 'MySQL5.7.26 已启动', 'MySQL5.7.26 正在启动...'). At the bottom right, it says '版本: 8.1.1.3'.

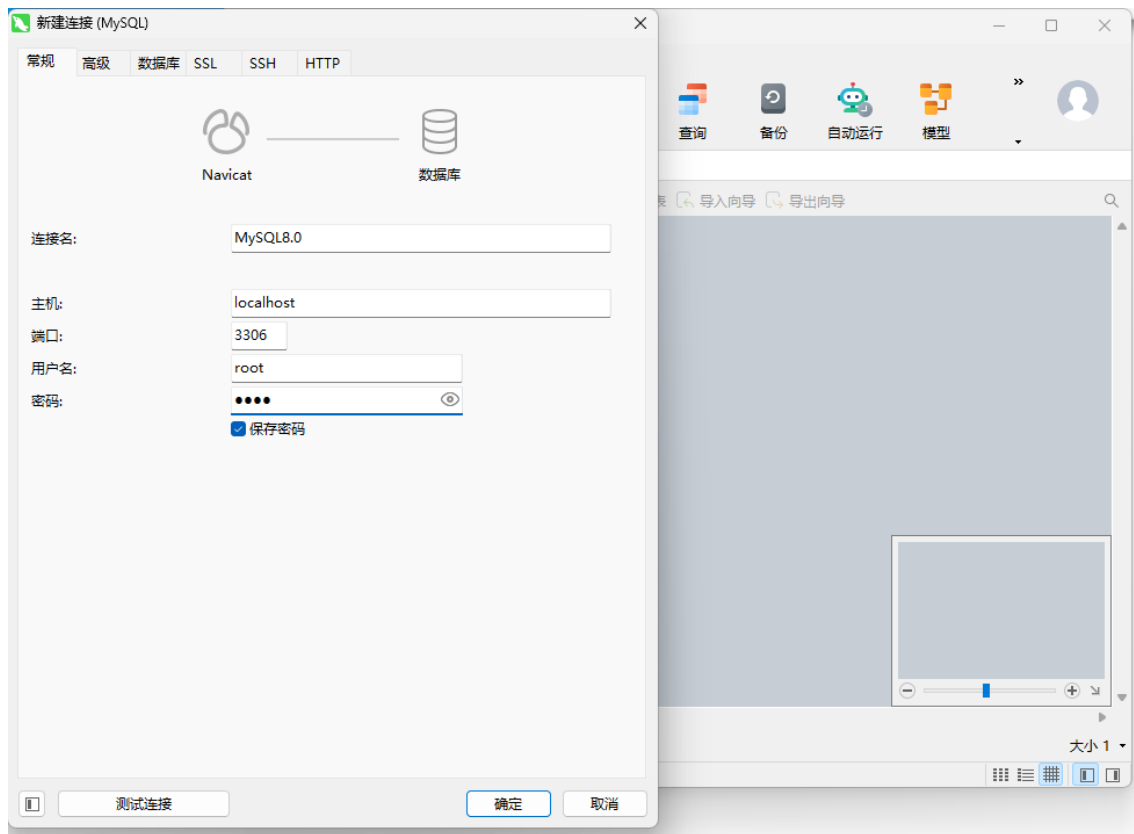
2. 在数据库管理中安装MySQL8.0



3. 启动MySQL8.0



4. 使用Navicat连接MySQL8



5. 验证MySQL版本

```
SELECT VERSION();
```

MySQL8新特性_事务性数据字典

MySQL8 的事务性数据字典是相比 MySQL 5.7 的核心改进之一，彻底改变了元数据（如表结构、视图、索引等）的存储和管理方式。

数据库的元数据包含表结构、视图、索引等等数据，在MySQL 5.7之前，元数据存储的文件系统中，这样的存储方式会导致DDL操作不支持事务，即无法做到批量的DDL操作**要么完全执行，要么完全不执行**。

我们以MySQL 5.7举例：

```
# 创建testdb数据库
CREATE DATABASE testdb;
# 使用数据库
use testdb;
# 创建t1表
CREATE TABLE t1(c1 int);
# 查看testdb库中的所有表
show TABLES;
# 删除t1,t2表
DROP TABLE t1,t2;
# 查看testdb库中的所有表
show TABLES;
```

我们会发现，虽然删除t1,t2表的语句执行失败了，但t1表仍然被删除，这就说明DDL的操作不支持事务。

而MySQL8之后，采用了InnoDB表的方式存储元数据，这种方式支持事务处理，设计目标是为处理大容量数据发挥最大化性能。所以MySQL8的DDL操作是支持事务的。我们再使用MySQL8执行同样的操作：

```
# 创建testdb数据库
CREATE DATABASE testdb;
# 使用数据库
use testdb;
# 创建t1表
CREATE TABLE t1(c1 int);
# 查看testdb库中的所有表
show TABLES;
# 删除t1,t2表
DROP TABLE t1,t2;
# 查看testdb库中的所有表
show TABLES;
```

我们发现，删除t1,t2表的语句执行失败了，t1表没有被删除，这说明MySQL8的DDL的操作是支持事务的。

除了支持DDL事务，元数据 InnoDB 的存储方式还会在数据库服务崩溃后利用 InnoDB 日志，快速恢复数据，使其在生产环境下更加稳定。并且元数据的查询效率也要比 MySQL5.7 更高，对于大规模、高并发场景，数据库的扩展性也更强。

实时效果反馈

1. 以下说法正确的是：

- ☒ A MySQL5.7的DDL语句支持事务，MySQL8.0的DDL语句不支持事务
- ☐ B MySQL5.7的DDL语句不支持事务，MySQL8.0的DDL语句支持事务
- ☐ C MySQL5.7和MySQL8.0的DDL语句都不支持事务
- ☐ D MySQL5.7和MySQL8.0的DDL语句都支持事务

答案

1=>B

MySQL8新特性_默认字符集和排序规则

MySQL 8.0 将默认字符集从 MySQL 5.7 的 `latin1` 更改为 `utf8mb4`，并将默认排序规则从 `latin1_swedish_ci` 更改为 `utf8mb4_0900_ai_ci`。这一变化显著提升了字符集支持的范围和排序精度。

`utf8mb4` 是 MySQL 对 UTF-8 的实现，支持完整的 Unicode 字符集，适合存储多语言文本、Emoji 和特殊符号。包括多国语言和补充语言，如 Emoji 和罕见汉字。而 `latin1` 仅支持 256 个字符，无法存储 Emoji 或汉字。

而 `utf8` 的排序规则 `utf8mb4_0900_ai_ci` 基于 Unicode 9.0 标准，取代了 MySQL 5.7 中的 `utf8` 排序规则 `utf8_general_ci`，提供更精确的 Unicode 排序，比较和排序更符合语言习惯（如中文拼音、笔画排序）。

MySQL 5.7 也可以设置使用字符集 `utf8mb4`，只不过默认字符集是 `latin1`，但它的 `utf8` 排序规则只能是 `utf8_general_ci`，不能使用 MySQL 8.0 的 `utf8mb4_0900_ai_ci`

我们来演示一下在 MySQL 8.0 中存储 Emoji 和汉字：

```
-- 创建表，显式指定字符集和排序规则
CREATE TABLE test_emoji (
  id INT PRIMARY KEY,
  content VARCHAR(100) CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci
);

-- 插入包含 Emoji 的数据
INSERT INTO test_emoji (id, content) VALUES (1, '😊 你好，MySQL 8.0!');

-- 查询数据
SELECT content FROM test_emoji WHERE id = 1;
```

实时效果反馈

1. MySQL 8.0 的默认字符集是

- ☐ A latin1
- ☐ B utf8
- ☐ C utf8mb4
- ☐ D gbk

答案

1=>C

将MySQL8的数据库导入到MySQL5.7中



我是 DeepSeek，很高兴见到你！

我可以帮你写代码、读文件、写作各种创意内容，请把任务交给我吧~

仅识别附件中的文字



testdb.sql
SQL 1005B

这是我从MySQL8中导出的SQL文件，现在我要把它导入到MySQL5.7中，帮我修改这个文件，完成导入

深度思考 (R1)

联网搜索



在实际开发过程中，我们有时会修改数据库版本，或者将别人的数据库内容导入自己的MySQL中。如果我们导出的数据库版本是MySQL8，现在要将数据导入MySQL5.7当中，往往会出现导入异常。

这大概率就是由于MySQL5.7和MySQL8.0的字符集排序规则和源数据存储方式不同，我们需要打开SQL文件，将建表语句中的字符集和排序规则变成MySQL5.7的规则：

- 将 `COLLATE utf8mb4_0900_ai_ci` 替换为 `COLLATE utf8mb4_general_ci`。
- 将 `ENGINE = InnoDB` 替换为 `ENGINE = MyISAM`
- 去掉表定义末尾的 `AUTO_INCREMENT = 1`

```
SET NAMES utf8mb4;
SET FOREIGN_KEY_CHECKS = 0;

-- Table structure for test_emoji
--
DROP TABLE IF EXISTS `test_emoji`;
CREATE TABLE `test_emoji` (
  `id` int(11) NOT NULL,
  `content` varchar(100) CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci NULL DEFAULT NULL,
  PRIMARY KEY (`id`) USING BTREE
) ENGINE = MyISAM AUTO_INCREMENT = 1 CHARACTER SET = utf8 COLLATE = utf8_general_ci ROW_FORMAT = Dynamic;

-- Records of test_emoji
--
INSERT INTO `test_emoji` VALUES (1, '☺ 你好, MySQL 8.0!');

SET FOREIGN_KEY_CHECKS = 1;
```

当然，由于在真正开发过程中，数据表的数量很庞大，我们手动进行修改免不了会出现问题，所以这里我更建议让人工智能帮我们进行修改：



我是 DeepSeek，很高兴见到你！

我可以帮你写代码、读文件、写作各种创意内容，请把你的任务交给我吧~

仅识别附件中的文字



testdb.sql
SQL 1005B

这是我从MySQL8中导出的SQL文件，现在我要把它导入到MySQL5.7中，帮我修改这个文件，完成导入

深度思考 (R1)

联网搜索



若要将MySQL5.7的数据导入MySQL8的服务中，一般情况下不会出问题，如果导入失败，我们也可以使用AI帮我们将语句修改为符合MySQL8的规则

实时效果反馈

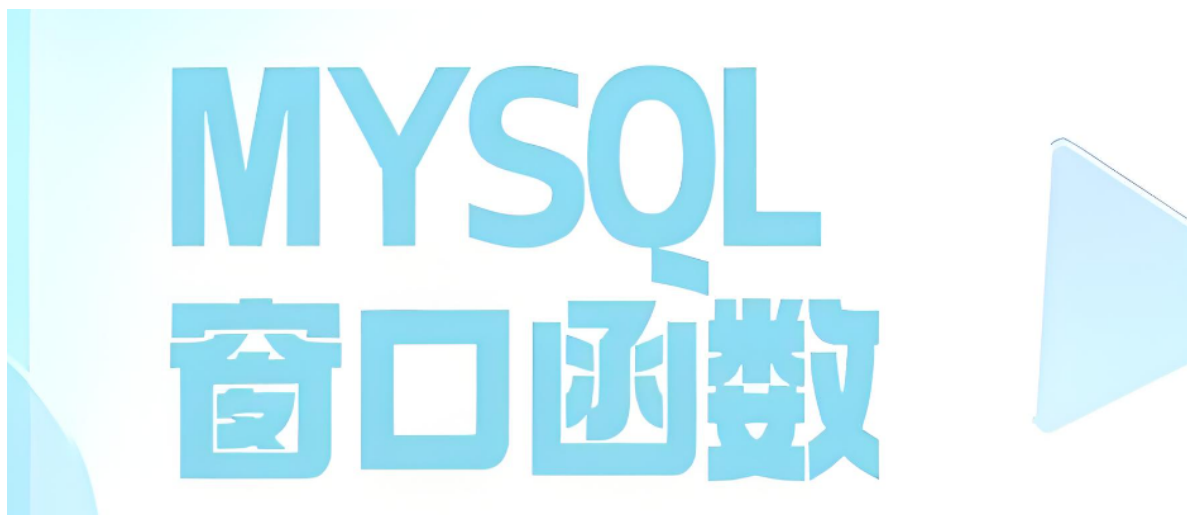
1. MySQL8的数据导入MySQL5.7的服务中，SQL文件中需要修改的是

- A 将 `COLLATE utf8mb4_0900_ai_ci` 替换为 `COLLATE utf8mb4_general_ci`
- B 将 `CHARACTER SET utf8` 替换为 `CHARACTER SET utf8mb4`。
- C 将 `COLLATE utf8_general_ci` 替换为 `COLLATE utf8mb4_0900_ai_ci`。
- D 不需要修改

答案

1=>A

MySQL8新特性_窗口函数



介绍

窗口函数也叫**分析函数**，用于将**查询的结果**进行较为复杂的统计分析，MySQL从8.0开始支持窗口函数。

假设我现在有这样一张表：

id	name	department	sales_volume	statistical_date
1	张三	销售1部	8000	2030-01-31
2	张三	销售1部	9000	2030-02-28
3	张三	销售1部	10000	2030-03-31
4	张三	销售1部	11000	2030-04-30
5	张三	销售1部	10000	2030-05-31

我想要每个月都统计张三今年的**销售总额**。也就是这样的一个效果：

id	name	departmen	sales_volume	statistical_date	sum
1	张三	销售1部	8000	2030-01-31	8000
2	张三	销售1部	9000	2030-02-28	17000
3	张三	销售1部	10000	2030-03-31	27000
4	张三	销售1部	11000	2030-04-30	38000
5	张三	销售1部	10000	2030-05-31	48000

此时使用原来的SQL语句，是非常难做到这样的效果的，而要想对**查询结果**进行复杂的分析处理，最好的方式就是使用**窗口函数**。

窗口函数中引入了一个**窗口**的概念，窗口类似于窗户，限定一个空间范围，或者可以理解为查询结果的自定义集合。而窗口函数可以在处理查询结果的时候，**每条结果**都可以结合**不同的窗口**，从而输出相应的数据分析结果。对窗口内数据**不同的处理方法**，可以使用**不同的窗口函数**。例如：我们可以查询张三所有月份中**最高的销售额**

id	name	departmen	sales_volume	statistical_date	max
1	张三	销售1部	8000	2030-01-31	11000
2	张三	销售1部	9000	2030-02-28	11000
3	张三	销售1部	10000	2030-03-31	11000
4	张三	销售1部	11000	2030-04-30	11000
5	张三	销售1部	10000	2030-05-31	11000

注：此时不同记录使用的窗口是一样的。

格式

窗口函数语法规则:

<窗口函数> Over

(

[Partition By] <用于分组的列名>

[Order By] <用于排序的列名>

(Rows | Range Between <上限> And <下限>)

)

窗口函数不能过滤查询结果，只能对查询结果进行**分析并输出**。所以窗口函数是写在SELECT之后，而不是写在WHERE之后。即：

```
SELECT 窗口函数 where 条件
```

而窗口函数的基本格式如下：

```
函数名(字段) over(子句)
```

其中over是关键字，用来指定函数执行的窗口范围，包含三个分析子句：分组(partition by)子句，排序(order by)子句，窗口(rows)子句，如果后面括号中什么都不写，则意味着窗口包含所有的查询结果，窗口函数的完整格式如下：

```
函数名(字段) over(  
    partition by <要分组的字段>  
    order by <排序的字段>  
    rows <窗口范围>  
)
```

我们来举个例子：

1. 向MySQL8数据库中导入销售额表
2. 查询每个人的销售额，并统计每个月末，每个人的**销售总额**。

```
SELECT  
    *,  
    sum( sales.sales_volume ) over ( -- 计算sales_volume字段的和  
        PARTITION BY name -- 根据名字分组  
        ORDER BY YEAR(sales.statistical_date),MONTH(sales.statistical_date)  
        -- 根据月份排序  
        rows between unbounded preceding and current row -- 窗口范围从之前的所有  
        行到当前行  
    ) sum  
FROM  
    sales
```

接下来我们具体介绍一下rows子句的关键字和写法：

- **preceding**：前面行
- **following**：后面行
- **current row**：当前行
- **unbounded**：所有行

例如：

- rows between 2 preceding and current row：窗口取前面两行到当前行
- rows between unbounded preceding and current row：窗口取之前所有行到当前行
- rows between current row and unbounded following：窗口当前行到之后所有行
- rows between 3 preceding and 1 following：窗口取当前行的前面三行到后面一行，总共五行

如果子句中只有partition by没有order by和rows，窗口规范默认是所有行

如果子句中有order by没有rows，窗口规范默认是取之前所有行到当前行

所以之前的例子也可以这么写：

```
SELECT
    *,
    sum( sales.sales_volume ) over ( -- 计算sales_volume字段的和
        PARTITION BY name -- 根据名字分组
        ORDER BY YEAR(sales.statistical_date),MONTH(sales.statistical_date)
        -- 根据月份排序
    ) sum
FROM
    sales
```

实时效果反馈

1. 窗口函数使用关键字定义窗口范围

A PARTITION BY

B ORDER BY

C ROWS

D GROUP BY

2. 窗口函数使用关键字定义排序方式

A PARTITION BY

B ORDER BY

C ROWS

D GROUP BY

答案

1=>C 2=>B

接下来我们分类介绍窗口函数：

聚合函数

窗口函数中的聚合函数，负责在窗口内执行聚合计算，支持窗口内累计或统计。常见的聚合函数有这几种：

- **SUM()**：计算窗口内值的总和
- **AVG()**：计算窗口内值的平均值
- **COUNT()**：计算窗口内的行数
- **MIN()**：找出窗口内的最小值
- **MAX()**：找出窗口内的最大值

接下来我们用查询语句举例聚合函数的使用：

```
-- 计算每个销售人员的累计销售额
SELECT
    *,
    SUM(sales_volume) OVER(PARTITION BY name ORDER BY statistical_date) AS
    running_total
FROM sales;

-- 计算每个部门的平均销售额
SELECT
    *,
    AVG(sales_volume) OVER(PARTITION BY department) AS dept_avg_sales
FROM sales;

-- 计算每个销售人员的销售记录数量
SELECT
    *,
    COUNT(*) OVER(PARTITION BY name) AS sales_count
FROM sales;

-- 找出每个销售人员的最低单月销售额
SELECT
    *,
    MIN(sales_volume) OVER(PARTITION BY name) AS min_sales
FROM sales;

-- 找出每个销售人员的最高单月销售额
SELECT
    *,
    MAX(sales_volume) OVER(PARTITION BY name) AS max_sales
FROM sales;
```

SQL中聚合函数，与窗口函数中聚合函数有什么区别：

我们以同样的需求举例：

```
-- 找出每个销售人员的最高单月销售额
SELECT
    *,
    MAX(sales_volume) OVER(PARTITION BY name) AS max_sales
FROM sales;

-- 找出每个销售人员的最高单月销售额
SELECT name, SUM(sales_volume) FROM sales GROUP BY name;
```

- 普通场景下的聚合函数基于GROUP BY子句定义的组进行计算，将多条记录聚合为一条，只能展示分组字段和聚合函数字段；
- 窗口函数是基于窗口定义进行计算，每条记录都会执行，而且针对于不同的记录可以设置不同的窗口范围，有几条记录执行完还是几条。

实时效果反馈

1. 窗口函数中，计算窗口内值的总和的函数是

- ☒ A SUM()
- ☐ B AVG()
- ☐ C ROW_NUMBER()
- ☐ D RANK()

2. 窗口函数中，计算窗口内值的平均值的函数是

- ☒ A SUM()
- ☐ B AVG()
- ☐ C ROW_NUMBER()
- ☐ D RANK()

答案

1=>A 2=>B

排名函数

排名函数可以为数据集中的行分配排名或序号，常用于排序和比较场景。常见的排名函数有这几种：

- **ROW_NUMBER()**：为结果集中的每一行分配唯一的连续序号（1, 2, 3...），即使值相同也会分配不同序号
- **RANK()**：为行分配排名，相同值获得相同排名，但会跳过后续排名（如1,1,3）
- **DENSE_RANK()**：类似RANK()但不跳过排名数字（如1,1,2）

接下来我们用查询语句举例排名函数的使用：

```
-- 计算3月销售额排名
SELECT
    *,
    ROW_NUMBER() OVER(ORDER BY sales_volume DESC) AS rn,
    RANK() OVER(ORDER BY sales_volume DESC) AS r,
    DENSE_RANK() OVER(ORDER BY sales_volume DESC) AS dr
FROM sales
WHERE MONTH(statistical_date) = 3;
```

实时效果反馈

1. 窗口函数中，计算窗口内值排名的函数是

- A** SUM()
- B** AVG()
- C** ROW_NUMBER()
- D** FIRST_VALUE()

答案

1=>C

分析函数

分析函数用于访问窗口内其他行的数据。常见的分析函数有这几种：

- **LAG(column, n, default)**: 返回当前行前 n 行的 column 列的值，若没有则返回当前行中default列的值。
- **LEAD(column, n, default)**: 返回当前行后 n 行的 column 值，若没有则返回当前行中default列的值。
- **FIRST_VALUE(column)**: 返回窗口内第一行的 column 值。
- **LAST_VALUE(column)**: 返回窗口内最后一行的 column 值。
- **NTH_VALUE(column, n)**: 返回窗口内第 n 行的 column 值。

接下来我们举例分析函数的使用：

```
-- 显示每个销售人员上个月的销售额
-- 显示每个销售人员下个月的销售额
-- 显示每个销售人员第一个月的销售额
-- 显示每个销售人员第二个月的销售额
-- 显示每个销售人员最后一个月的销售额
SELECT
    *,
    LAG(sales_volume, 1) OVER(
        PARTITION BY name
        ORDER BY statistical_date
        rows BETWEEN unbounded preceding AND unbounded following
    ) AS prev_month_sales,
    LEAD(sales_volume, 1, sales_volume) OVER(
```

```

        PARTITION BY name
        ORDER BY statistical_date
        rows BETWEEN unbounded preceding AND unbounded following
    ) AS next_month_sales,
    FIRST_VALUE(sales_volume) OVER(
        PARTITION BY name
        ORDER BY statistical_date
        rows BETWEEN unbounded preceding AND unbounded following
    ) AS first_month_sales,
    NTH_VALUE(sales_volume,2) OVER(
        PARTITION BY name
        ORDER BY statistical_date
        rows BETWEEN unbounded preceding AND unbounded following
    ) AS second_month_sales,
    LAST_VALUE(sales_volume) OVER(
        PARTITION BY name
        ORDER BY statistical_date
        rows BETWEEN unbounded preceding AND unbounded following
    ) AS last_month_sales
FROM sales;

```

实时效果反馈

1. 窗口函数中，返回窗口内第一行的值的函数是

- A** PERCENT_RANK()
- B** LAST_VALUE()
- C** FIRST_VALUE()
- D** NTH_VALUE()

答案

1=>C

分布函数

分布函数计算行在窗口内的相对位置或分布。

CUME_DIST(): 返回当前行值在窗口内的累积分布 ($0 < \text{值} \leq 1$)。

PERCENT_RANK(): 返回当前行在窗口内的百分比排名 ($0 \leq \text{值} < 1$)。

接下来我们举例分布函数的使用：

```

-- 计算1月每个销售人员销售额的百分比排名
SELECT
    *,
    PERCENT_RANK() OVER(ORDER BY sales_volume DESC) AS pr,
    CUME_DIST() OVER(ORDER BY sales_volume DESC) AS cd
FROM sales WHERE MONTH(statistical_date) = 1;

```

实时效果反馈

1. 窗口函数中，当前行在窗口内的百分比排名的函数是

A PERCENT_RANK()

B LAST_VALUE()

C FIRST_VALUE()

D NTH_VALUE()

答案

1=>A

MySQL8新特性_通用表表达式

介绍

通用表表达式(Common Table Expressions, CTE)是MySQL 8.0引入的一项重要功能，它极大地增强了SQL查询的**可读性**和**灵活性**。

CTE是一个临时的结果集，只在查询执行期间存在，它可以看作是一个临时表，我们经常使用CTE代替复杂的子查询语句，增强SQL的可读性。基本语法为：

```
WITH cte_name AS (  
    SELECT ... -- CTE查询定义  
)  
SELECT * FROM cte_name; -- 主查询
```

接下来我们举个例子，分别使用子查询和CTE完成：

```
# 找出每月销售额超过该部门当月平均销售额的销售人员  
# 子查询  
SELECT *  
FROM sales s  
JOIN(  
    SELECT  
        department,  
        MONTH(statistical_date) AS month,  
        AVG(sales_volume) AS avg_sales  
    FROM sales  
    GROUP BY department,MONTH(statistical_date)  
) AS dept_avg  
ON s.department = dept_avg.department AND  
MONTH(s.statistical_date)=dept_avg.month  
WHERE s.sales_volume > dept_avg.avg_sales;  
  
# CTE  
WITH dept_avg AS(  
    SELECT  
        department,  
        MONTH(statistical_date) AS month,  
        AVG(sales_volume) AS avg_sales  
    FROM sales  
    GROUP BY department,MONTH(statistical_date)
```

```

SELECT
    department,
    MONTH(statistical_date) AS month,
    AVG(sales_volume) AS avg_sales
FROM sales
GROUP BY department, MONTH(statistical_date)
)

SELECT *
FROM sales s
JOIN dept_avg
ON s.department = dept_avg.department AND
MONTH(s.statistical_date)=dept_avg.month
WHERE s.sales_volume > dept_avg.avg_sales;

```

CTE的优势：

1. 分步清晰：将复杂查询分解为逻辑步骤
2. 可读性强：每个CTE块有明确的业务含义
3. 易于维护：修改一个步骤不影响其他部分
4. 避免重复：每个子查询只需定义一次
5. 调试方便：可以单独测试每个CTE块

实时效果反馈

1. MySQL8的通用表表达式，相比于子查询，优势是

- ☒ A 可读性强
- ☐ B 调试方便
- ☐ C 避免重复
- ☐ D 以上都正确

答案

1=>D

案例

接下来我们完成一个CTE的案例：

```

-- 输出每个月每个人的销售额，以及该部门该月的平均销售额，以及该员工每个月的平均销售额
WITH
-- 部门月平均销售额
dept_avg AS (
    SELECT
        department,
        MONTH(statistical_date) AS month,
        AVG(sales_volume) AS avg_sales

```

```

        FROM sales
        GROUP BY department,MONTH(statistical_date)
    ),
    -- 员工平均销售额
    emp_avg AS (
        SELECT
            name,
            AVG(sales_volume) AS avg_sales
        FROM sales
        GROUP BY name
    )

SELECT
    sales.*,
    dept_avg.avg_sales AS 部门该月平均销售额,
    emp_avg.avg_sales AS 个人销售均值
FROM
    sales
LEFT JOIN dept_avg
ON sales.department = dept_avg.department AND MONTH(sales.statistical_date) =
dept_avg.month
LEFT JOIN emp_avg
ON sales.name = emp_avg.name

```

MySQL8新特性_正则表达式函数

MySQL 8.0 提供了强大、灵活的正则表达式函数，支持复杂的模式匹配、替换、提取和定位操作。新增的函数主要有以下几个：

1. **REGEXP_LIKE(expr, pattern, [match_type])**：检查字符串是否匹配正则表达式。
 - expr：要检查的字符串（列或常量）。
 - pattern：正则表达式。
 - match_type（可选）：匹配选项
 - c：大小写敏感（默认）。
 - i：忽略大小写。
2. **REGEXP_REPLACE(expr, pattern, replacement)**：替换匹配正则表达式的子字符串，将字符串 expr 中匹配 pattern 的部分替换为 replacement。
3. **REGEXP_SUBSTR(expr, pattern)**：提取 expr 中匹配 pattern 的子字符串。
4. **REGEXP_INSTR(expr, pattern)**：返回 pattern 在 expr 中匹配的起始位置。

接下来我们就使用几个查询案例演示正则表达式函数的用法：

```

-- 创建用户信息表
CREATE TABLE contacts (
    id INT PRIMARY KEY,
    name VARCHAR(50),
    email VARCHAR(100),
    phone VARCHAR(20)
);

```

```

-- 插入数据
INSERT INTO contacts (id, name, email, phone) VALUES
(1, 'Alice', 'alice123@example.com', '123-456-7890'),
(2, 'Bob', 'bob.smith@domain.org', '(987) 654-3210'),
(3, 'Charlie', 'invalid.email', '+1-555-555-5555'),
(4, 'David', 'david_99@company.co.uk', '1234567890');

-- 筛选有效邮箱地址（格式：用户名@域名.顶级域名）
SELECT *
FROM contacts
WHERE REGEXP_LIKE(email, '^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$',
'i');

-- 将电话号码替换为统一格式（无标点符号）
SELECT *,
    REGEXP_REPLACE(phone, '[^0-9]', '') AS cleaned_phone
FROM contacts;

-- 从邮箱中提取域名部分（@ 后面的内容）
SELECT *,
    REGEXP_SUBSTR(email, '@[a-zA-Z0-9.-]+') AS domain
FROM contacts;

-- 查找邮箱中 @ 符号的起始位置。
SELECT *,
    REGEXP_INSTR(email, '@') AS at_position
FROM contacts;

```

实时效果反馈

1. MySQL8中，检查字符串是否匹配正则表达式的函数是

- A** REGEXP_LIKE
- B** REGEXP_REPLACE
- C** REGEXP_SUBSTR
- D** REGEXP_INSTR

答案

1=>A

